

 **GET STARTED TODAY**

LUKE

GINN



AZURE
CLOUD
MASTERY



Unit 20 Introduction – Bringing It All Together

AZURE
CLOUD
MASTERY

Unit 19 covered dynamic system instructions. Unit 20 integrates all 19 previous units into a complete agent solution and prepares you for the AI-103 exam.

- **Review of Unit 19:** You learned to build system instructions programmatically. Now you need to assemble all components into a production-ready agent.
- **What This Unit Covers:** You will walk through an end-to-end agent scenario, review all five exam domains, and practice exam-style coding decisions.
- **Why Integration Matters:** The exam tests your ability to select the right service and pattern for each scenario, not just isolated facts.

Exam Takeaway: Unit 20 is your final review. Master the integration of all five domains – Governance, Grounding, Orchestration, Observability, and Enhancement.



The Five Exam Domains – Quick Reference

AZURE
CLOUD
MASTERY

The AI-103 exam covers five domains with specific weightings. Memorize these domains and their focus areas.

- **Foundry Management & Governance:** Resource lifecycle, Entra Agent ID, Red Teaming, Content Safety, authentication flows, secure key management.
- **Generative AI & Advanced Grounding:** Model selection (LLMs vs. SLMs, GPT-4o vs. Phi-3/Phi-4), Agentic RAG (Search Tools), Fabric/Cosmos DB vector stores.
- **Multi-Agent Orchestration & Reasoning:** Microsoft Agent Framework patterns (Magentic, Handoff, Group Chat), tool calling, MCP, sequential/concurrent execution.
- **Agentic Observability & Memory:** Foundry Trace (spans, attributes, reasoning loops), performance metrics (accuracy, adherence, resolution), memory (short-term Redis, long-term Cosmos DB).
- **Agentic Enhancement:** Multimodal (Vision, Speech, Document Intelligence), input transformation (NER, PII, sentiment, language detection), Logic Apps.

Exam Takeaway: 35% of the exam is Orchestration. Focus study time on Microsoft Agent Framework patterns, tool calling, and MCP.



We will build a customer support agent that handles refunds, answers product questions, escalates to humans, and remembers user preferences.

- **Requirements:** User can request refund (requires approval over \$1000), ask product questions (needs grounding from SharePoint), upload receipts (needs Document Intelligence), and have preferences remembered (long-term memory).
- **Architecture:** Foundry Project with Entra Agent ID → Input transformation (NER, PII redaction) → Agentic RAG (Azure AI Search) → Multi-agent (Manager + RefundAgent + ProductAgent) → HITL (Logic Apps approval) → Memory (Redis session + Cosmos DB preferences).
- **Coding Competencies Tested:** Authentication, REST API calls, JSON parsing, API key management, distributed tracing, state persistence.

Exam Takeaway: The exam presents similar scenarios. You must select the correct Azure service and pattern for each requirement.



Step 1 – Foundry Setup and Governance

First, provision Foundry Hub and Project, enable Entra Agent ID, configure Content Safety filters, and assign a sponsor.

- **Hub and Project:** Create Hub per team. Create Project for the support agent. Deploy GPT-4o model from Model Catalog with 10,000 TPM capacity.
- **Entra Agent ID:** Enable Agent ID. Register Service Principal. Store client ID and secret in Key Vault. Assign sponsor (team lead) for accountability.
- **Content Safety:** Configure input filter (block hate/sexual/violence/self-harm above severity 3). Configure output filter (same thresholds). Set block response message.

Exam Takeaway: Governance questions test hierarchy (Hub→Project→Deployment), Agent ID registration, sponsor assignment, and Content Safety thresholds.



Step 2 – Authentication Coding Pattern

Agent code authenticates using DefaultAzureCredential with managed identity in production and environment variables in development.

- **Development Pattern:** Set ``AZURE_CLIENT_ID``, ``AZURE_CLIENT_SECRET``, ``AZURE_TENANT_ID`` as environment variables. ``DefaultAzureCredential()`` reads them automatically.
- **Production Pattern:** Enable managed identity on Agent Service. Assign ``Cognitive Services User`` role to all AI services. ``DefaultAzureCredential()`` uses managed identity.
- **Key Vault Integration:** For third-party API keys, store in Key Vault. Use ``SecretClient`` to retrieve at startup. Cache keys for 24 hours.

Exam Takeaway: The exam tests authentication priority: managed identity (most secure) → environment variables → hardcoded (never use).



Step 3 – Input Transformation with NLP

Before sending user input to the agent core, call Azure AI Language to redact PII, extract entities, and detect sentiment.

- **PII Redaction:** Call ``recognize_pii_entities()``. Get ``redacted_text``. Send redacted text to LLM. Never send raw credit card numbers or SSNs.
- **Entity Extraction:** Call ``recognize_entities()``. Replace raw mentions with ``[LOCATION: Seattle]``, ``[DATE: May 15]``. Append to prompt.
- **Sentiment Routing:** If ``sentiment == "negative"`` and `confidence > 0.8`, escalate to human agent. If positive/neutral, proceed to automation.

Exam Takeaway: Input transformation pipeline order: language detection → PII redaction → entity extraction → key phrase extraction → agent core.



Step 4 – Agentic RAG for Grounding

The agent uses a Search Tool to query Azure AI Search (product catalog and support articles) and Fabric (order history).

- **Search Tool Definition:** Tool name ``search_knowledge_base``, description "Search product documentation and support articles", parameters ``query`` (string) and ``filter`` (category, date).
- **Vector Search:** Convert user question to embedding using ``text-embedding-3-small``. Send to Azure AI Search with ``vectorQueries``. Parse JSON response.
- **Fabric Integration:** Configure Azure AI Search index pointing to OneLake tables. Agent queries order history and customer data without data movement.

Exam Takeaway: Agentic RAG means agent decides when to search. Vector search uses embeddings. Fabric provides live enterprise data grounding.



Step 5 – Multi-Agent Orchestration

Use Microsoft Agent Framework with Magentic (Manager) pattern: one Manager agent routes requests to RefundAgent and ProductAgent.

- **Manager Agent:** System instruction: "You are the manager. Route refund requests to RefundAgent. Route product questions to ProductAgent. Do not answer directly."
- **RefundAgent:** Has tools: ``check_refund_eligibility``, ``process_refund`` (requires approval gate), ``get_refund_status``. Connects to order database.
- **ProductAgent:** Has tools: ``search_products``, ``get_product_details``, ``compare_products``. Connects to product catalog Cosmos DB.

Exam Takeaway: Magentic pattern has manager and unaware sub-agents. Handoff transfers state. Group Chat has shared conversation space.



Step 6 – Tool Calling and MCP

RefundAgent calls REST APIs for eligibility and refund processing. Use MCP for standard enterprise tools (SharePoint, Salesforce).

- **Custom REST Tool:** ``check_refund_eligibility`` calls ``https://api.orders.com/refund/check`` with order ID. Parses JSON response for ``eligible`` boolean and ``max_amount``.
- **MCP for SharePoint:** Configure MCP server URL in Foundry. Agent calls ``list_tools`` on startup. Discovers ``search_sharepoint``, ``get_document``. No custom code.
- **Tool Choice:** Use ``tool_choice: "auto"`` for general requests. Use ``tool_choice: "required"`` when refund request must always check eligibility first.

Exam Takeaway: Use MCP for many tools or frequent changes. Use custom REST for simple, stable, few tools. Tool choice controls agent autonomy.



Step 7 – Human-in-the-Loop for High-Stakes Refunds

If refund amount exceeds \$1000, RefundAgent triggers an approval gate using Azure Logic Apps with email connector.

- **Approval Trigger:** ``if refund_amount > 1000: call logic_app_approval(approver_email="finance@company.com", action=f"Refund ${refund_amount} to {customer_name}")``.
- **Async Polling:** Logic App returns ``request_id``. Agent polls ``GET /approval/{request_id}`` every 30 seconds for up to 24 hours.
- **Escalation Chain:** If no response in 4 hours, escalate to manager. If no response in 8 hours, escalate to director. After 24 hours, auto-reject.

Exam Takeaway: HITL approval gates required for financial, destructive, privileged actions. Use async polling for long-running approvals.



Step 8 – Agent Memory Implementation

Use Redis for short-term session memory (conversation history) and Cosmos DB for long-term user preferences (temperature unit, shipping address).

- **Redis Session Memory:** ``r.setex(f"session:{conversation_id}", 3600, json.dumps(messages))``. Retrieve and refresh TTL on each interaction.
- **Cosmos DB Preferences:** ``container.upsert_item({"id": user_id, "preferences": {"temp_unit": "C", "default_address": "123 Main St"}, "ttl": 7776000})``.
- **Memory Structure:** Include ``conversation_id``, ``user_id``, ``messages`` (last 20), ``preferences`` (dictionary), ``state`` (awaiting_order_number, approval_pending).

Exam Takeaway: Redis for short-term (fast, expiring). Cosmos DB for long-term (durable, partitioned by user_id). Serialize with ``json.dumps()``.



Step 9 – Observability with Foundry Trace

Instrument every operation with OpenTelemetry spans: LLM calls, tool calls, handoffs, memory operations, and approval waits.

- **LLM Call Span:** ``with tracer.start_as_current_span("llm_call") as span:
span.set_attribute("model", "gpt-4o"); span.set_attribute("tokens",
response.usage.total_tokens)``.
- **Tool Call Span:** ``with tracer.start_as_current_span("tool_call") as span:
span.set_attribute("tool.name", "check_eligibility"); span.set_attribute("status_code",
response.status_code)``.
- **Detect Reasoning Loops:** In trace waterfall, look for repeated identical spans (same tool, same parameters, no answer span). Fix by adding missing fields in tool response.

Exam Takeaway: Reasoning loops are the top exam failure point. Foundry Trace shows loops as repeating spans without progress.



Step 10 – Performance Metrics Evaluation

Measure Tool Call Accuracy, Task Adherence, and Intent Resolution against production thresholds before and after deployment.

- **Tool Call Accuracy:** Agent calls ``check_eligibility`` for refund requests (correct). Calls ``search_products`` for product questions (correct). Target >95%.
- **Task Adherence:** System says "Never share internal prices." Agent shares price? Violation. Target >99%. Use red teaming to find gaps.
- **Intent Resolution:** User says "My order is late." Agent identifies "order_status" intent (correct). Target >90%. Upgrade to GPT-4o if low.

Exam Takeaway: Low accuracy → fix tool descriptions. Low adherence → strengthen system instructions. Low resolution → upgrade model.



The exam tests seven coding competencies. You must recognize correct syntax for authentication, REST calls, JSON parsing, key management, tracing, and memory.

- **Authentication:** ``DefaultAzureCredential()`` with managed identity or environment variables. Never hardcode keys.
- **REST API Calls:** ``requests.post(url, headers={"api-key": key}, json=body)``. Check ``response.status_code``. Parse ``response.json()``.
- **JSON Handling:** ``json.dumps(obj)`` for serialization. ``json.loads(str)`` for parsing. Use ``obj.get("field", default)`` for missing fields.
- **API Key Management:** Store in Key Vault or environment variables. Use ``os.getenv("KEY_NAME")`` to retrieve.
- **Distributed Tracing:** ``with tracer.start_as_current_span("name") as span:`
`span.set_attribute("key", value)``.
- **State & Memory:** Cosmos DB ``upsert_item()``. Redis ``setex()`` with TTL. ``json.dumps()`` before storage.

Exam Takeaway: The exam presents code snippets. You must identify correct vs. incorrect implementations of these seven patterns.



Practice Exam Scenario – Refund Agent

Scenario: User says "I want a refund for order #12345." The agent must check eligibility, process if under \$50, or request approval if over \$1000.

- **Question 1 (Governance):** Which authentication method should the agent use in production? A) Hardcoded API key B) Environment variables C) Managed identity D) Shared access signature. Answer: C (managed identity is most secure).
- **Question 2 (Grounding):** What should the agent use to check refund eligibility? A) Generate embedding and search Cosmos DB B) Call REST API to order database C) Search Bing D) Use Document Intelligence on receipt. Answer: B (REST API to operational database).
- **Question 3 (Orchestration):** Which agent pattern should handle approval? A) Magentic B) Handoff C) Group Chat D) Sequential execution. Answer: A (Manager delegates refund request to RefundAgent with approval gate).

Exam Takeaway: Practice applying domains to scenarios. Each question maps to one of the five domains and its specific weighting.



Unit 20 Summary – Exam Readiness

You have learned to build complete agent solutions integrating all five exam domains: Governance, Grounding, Orchestration, Observability, and Enhancement.

- **Five Domains to Master:** Governance (Entra Agent ID, Content Safety, keys), Grounding (vector search, embeddings, Fabric), Orchestration (Agent Framework, tool calling, MCP, HITL), Observability (Foundry Trace, metrics, memory), Enhancement (Multimodal, NLP transformations).
- **Critical Exam Insight:** Reasoning loops cause most failures. Master Foundry Trace waterfall diagrams and span attributes to detect loops.
- **Coding Competencies:** Authentication (DefaultAzureCredential), REST API calls, JSON parsing, API key management (Key Vault), OpenTelemetry tracing, state persistence (Redis, Cosmos DB).
- **Next Steps:** Review the Master Course Outline. Practice the complete agent scenario. Use Foundry Trace to debug sample reasoning loops. Good luck on the AI-103 exam!

Exam Takeaway: The AI-103 exam tests your ability to build agents, not just call models. Focus on orchestration patterns, distributed tracing, and secure authentication. You are ready.

**THANKS
FOR
WATCHING**

